📘 ManiChicks API Documentation

🌐 Base URL

`http://localhost:5000/api`

> 🔁 En production, remplacer par l'URL du serveur déployé.

🔐 Authentification

L'API utilise un mécanisme **JWT (JSON Web Token)**.

Chaque requête protégée doit inclure le header suivant :

`Authorization: Bearer <token>`

🔑 Login

POST `/auth/login`

Body

```
{
  "username": "admin",
  "password": "mdp_admin"
}
```

Response — `200 OK`

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "id": "65f1fg8dfd3c4d5e6fi8f7d8vd",
    "username": "admin",
    "email": "admin@manichicks.com",
    "firstName": "System",
    "lastName": "Administrator",
    "role": "admin"
  }
}
```

👤 Current User

GET `/auth/me`

Headers

```
Authorization: Bearer <token>
```

Response

```
{
  "success": true,
  "data": { /* user object */ }
}
```

📦 API Resources

🔒 Tous les endpoints nécessitent une authentification (sauf mention contraire).

🐔 1. Batches

📄 Get All Batches

GET `/batches`

Query Parameters

Param	Type	Description
status	string	`startup`, `growing`, `quarantine`, `ready_to_sell`, `all`
search	string	Recherche par nom

Example

```
GET /batches?status=growing
```

Response

```
{
  "success": true,
  "data": [ /* batch list */ ]
}
```

📊 Batch Statistics

GET `/batches/stats`

```
{
  "success": true,
  "data": {
    "totalBatches": 4,
    "growingBatches": 2,
    "readyBatches": 1,
    "totalBirds": 2750,
    "avgMortalityRate": "2.1"
  }
```

```
}
```

🔍 Get Batch By Code

GET `/batches/:batchCode`

➕ Create Batch

POST `/batches`
Access: `admin`, `manager`

```
{
  "name": "Bande B - Label Rouge",
  "supplierName": "Ferme du Sud",
  "buildingIds": ["65f1a2b3..."],
  "startDate": "2026-05-01",
  "initialCount": 800,
  "currentCount": 800
}
```

✏️ Update Batch

PUT `/batches/:batchCode`

```
{
  "status": "ready_to_sell",
  "avgWeightKg": 2.4
}
```

❌ Delete Batch

DELETE `/batches/:batchCode`
Access: `admin`

🏢 2. Buildings

📄 Get All Buildings

GET `/buildings`

```
{
  "success": true,
  "data": [ /* building list */ ]
}
```

🔍 Get Building

GET `/buildings/:buildingCode`

📡 Latest Sensor Data

GET `/buildings/:buildingCode/sensors/latest`

```
{
  "success": true,
  "data": [
    { "sensorType": "temperature", "value": 30.2, "timestamp": "2026-05-
02T14:30:00Z" }
  ]
}
```

➕ Create Building

POST `/buildings`
Access: `admin`, `manager`

```
{
  "name": "Bâtiment C",
  "type": "quarantine",
  "capacity": 200,
  "responsible": "Dr. Nganou"
}
```

✏️ Update Building

PUT `/buildings/:buildingCode`

❌ Delete Building

DELETE `/buildings/:buildingCode`
Access: `admin`

📡 3. Sensors (IoT)

➕ Add Sensor Data

POST `/sensors/data`

```
{
  "buildingCode": "BLD01",
  "sensorType": "temperature",
  "value": 30.5,
  "timestamp": "2026-05-02T10:00:00Z"
```

```
}
```

📈 Sensor History

GET `/sensors/history/:buildingCode`

Query Parameters

Param	Type	Description
hours	number	Default: `24`
type	string	`temperature`, `humidity`, `ammonia`

Example

```
GET /sensors/history/BLD01?hours=48&type=temperature
```

🩺 4. Health Monitoring

📄 Get All Records

GET `/health`

📄 Get Records By Batch

GET `/health/batch/:batchId`

➕ Create Record

POST `/health`
Access: `admin`, `veterinarian`

```
{
  "batchId": "65f1a2b3c4d5e6f7a8b9c0d1",
  "type": "exam",
  "veterinarian": "Dr. Kameni",
  "dailyMortality": 2
}
```

✏️ Update Record

PUT `/health/:recordCode`

❌ Delete Record

DELETE `/health/:recordCode`
Access: `admin`

💰 5. Sales

📄 Get All Sales

GET `/sales`

📊 Sales Statistics

GET `/sales/stats`

```
{
  "success": true,
  "data": {
    "totalRevenue": 1500000,
    "monthlyRevenue": []
  }
}
```

➕ Create Sale

POST `/sales`
Access: `admin`, `manager`

💳 Update Payment Status

PATCH `/sales/:saleCode/payment`

```
{
  "paymentStatus": "paid"
}
```

❌ Delete Sale

DELETE `/sales/:saleCode`
Access: `admin`

🚚 6. Suppliers

📄 Get All Suppliers

GET `/suppliers`

➕ Create Supplier

POST `/suppliers`

✏️ Update Supplier

PUT `/suppliers/:supplierCode`

❌ Delete Supplier

DELETE `/suppliers/:supplierCode`
Access: `admin`

📊 7. Dashboard

📊 Global Statistics

GET `/dashboard/stats`

🌡️ Temperature Trend (24h)

GET `/dashboard/temperature-trend`

⚠️ Error Handling

HTTP Status Codes

Code	Meaning
200	Success
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found

Code	Meaning
500	Server Error

❗ Error Response Format

```
{
  "success": false,
  "message": "Batch not found"
}
```

🔌 Frontend Integration (React)

🔑 Login Example

```
const login = async (username, password) => {
  const res = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username, password })
  });

  const data = await res.json();

  if (data.success) {
    localStorage.setItem('token', data.token);
  }

  return data;
};
```

📦 Authenticated Request

```
const fetchBatches = async () => {
  const token = localStorage.getItem('token');

  const res = await fetch('/api/batches', {
    headers: {
      Authorization: `Bearer ${token}`
    }
  });

  return res.json();
};
```

⚠️ Error Handling

```
if (!res.ok) {
  const error = await res.json();
  console.error(error.message);
}
```

📌 Additional Notes

- 📅 Format des dates : **ISO 8601** (`YYYY-MM-DDTHH:mm:ss.sssZ`)
- 🔑 Les identifiants (`batchCode`, `buildingCode`, etc.) sont générés côté backend
- 🧱 Les `_id` MongoDB sont utilisés pour les relations internes (`batchId`, etc.)

🚀 Support

Pour toute question technique : contacter l'équipe backend.